

LocalLLM Chat

Privacy-First Offline Android Application

with RAG, 3D Avatar, OCR & Firebase Integration

Prepared by:

Yassin CHAHBANE

Class: 4IIR 2DSI G1

Supervised by:

Dr. Mohammed BOUSMAH

Academic Year: 2025/2026

Dedications

*To my beloved parents,
whose endless support and sacrifices made this journey possible.*

*To my dear siblings,
for their constant encouragement.*

*To all my teachers,
who guided me throughout my academic path.*

*To my friends and colleagues,
for their companionship and motivation.*

Acknowledgments

*I would like to express my deepest gratitude to my supervisor,
Dr. Mohammed BOUSMAH, for his invaluable guidance,
patience, and encouragement throughout this project.*

*I am also thankful to all my professors in the 4IIR 2DSI G1 program
for providing me with the knowledge and skills necessary to
undertake this challenging project.*

*Special thanks to my family and friends for their unwavering support.
Their belief in me has been a constant source of motivation.*

Résumé

LocalLLM Chat: Application Android Privée Hors-Ligne

Ce projet présente LocalLLM Chat, une application Android innovante qui combine l'intelligence artificielle locale avec une architecture cloud hybride pour garantir à la fois la confidentialité des données et des fonctionnalités avancées. L'application permet aux utilisateurs de chatter avec un modèle de langage DeepSeek-R1 exécuté localement via Ollama, assurant qu'aucune donnée sensible ne quitte leur machine personnelle. Le système intègre également un mécanisme RAG (Retrieval-Augmented Generation) pour interroger des documents personnels hors-ligne, un avatar 3D rendu via WebView, une reconnaissance vocale, un scanner OCR pour l'extraction de texte à partir d'images, ainsi qu'une synchronisation cloud avec Supabase et des notifications push via Firebase. L'architecture est basée sur trois composants principaux : un client Android (Java), un backend d'authentification Spring Boot, et un service IA FastAPI orchestrant Ollama et ChromaDB. Les résultats démontrent la faisabilité d'un système d'IA conversationnelle respectueux de la vie privée, fonctionnant entièrement sur une infrastructure domestique.

Mots-clés : Android, LLM, RAG, Ollama, DeepSeek-R1, Spring Boot, FastAPI, Firebase, OCR, 3D Avatar, Confidentialité

Abstract

LocalLLM Chat: Private Offline Android Application

This project presents LocalLLM Chat, an innovative Android application that combines local artificial intelligence with hybrid cloud architecture to ensure both data privacy and advanced features. The application allows users to chat with a DeepSeek-R1 language model running locally via Ollama, ensuring that no sensitive data leaves their personal machine. The system also integrates a RAG (Retrieval-Augmented Generation) mechanism for querying personal documents offline, a 3D avatar rendered via WebView, voice recognition, an OCR scanner for text extraction from images, as well as cloud synchronization with Supabase and push notifications via Firebase. The architecture is based on three main components: an Android client (Java), a Spring Boot authentication backend, and a FastAPI AI service orchestrating Ollama and ChromaDB. The results demonstrate the feasibility of a privacy-respecting conversational AI system running entirely on a home infrastructure.

Keywords: Android, LLM, RAG, Ollama, DeepSeek-R1, Spring Boot, FastAPI, Firebase, OCR, 3D Avatar, Privacy

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CoT	Chain-of-Thought
CRUD	Create, Read, Update, Delete
CUDA	Compute Unified Device Architecture
FCM	Firebase Cloud Messaging
GLB	Graphics Library Binary
GLTF	Graphics Library Transmission Format
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP Secure
IA	Intelligence Artificielle (Artificial Intelligence)
IDE	Integrated Development Environment
JPA	Java Persistence API
JWT	JSON Web Token
LLM	Large Language Model
ML Kit	Machine Learning Kit
MVVM	Model-View-ViewModel
OCR	Optical Character Recognition
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language
VRAM	Video Random Access Memory

List of Figures

1.1	Project Gantt Diagram - 6-Week Development Timeline	12
2.1	High-Level System Architecture - Hybrid Local/Cloud Design	16
2.2	Use Case Diagram - System Actors and Interactions	16
2.3	Activity Diagram - Send Message with RAG Pipeline	17
2.4	Class Diagram - Android, Spring Boot & FastAPI Components	18

List of Tables

2.1	Functional Requirements	14
2.2	Non-Functional Requirements	15
3.5	Development Tools	21

Contents

General Introduction	10
1 General Project Context	11
Introduction	11
1.1 Project Presentation	11
1.1.1 Problem Statement	11
1.1.2 Proposed Solution	11
1.2 Work Methodology and Planning	12
1.2.1 Development Methodology	12
1.2.2 Gantt Diagram	12
Conclusion	13
2 Analysis and Design	14
Introduction	14
2.1 Functional Study	14
2.1.1 Analysis of Functional Requirements	14
2.1.2 Analysis of Non-Functional Requirements	15
2.2 Conceptual Study	15
2.2.1 Management Rules	15
2.2.2 UML Diagrams	16
Conclusion	18
3 Technical Study	19
Introduction	19
3.1 Project Architecture	19
3.1.1 Communication Layer	19
3.1.2 Architecture Workflow	19
3.1.3 Flow Rate Based Design	20
3.2 Technologies Used	20
3.2.1 Mobile Development	20
3.2.2 Backend AI Service (Python)	20
3.2.3 Authentication Backend (Java)	21
3.2.4 Cloud Services	21
3.3 Working Environment	21
Conclusion	21

4 Implementation	23
Introduction	23
4.1 Realization	23
4.1.1 Android Client Implementation	23
4.1.2 FastAPI RAG Pipeline Implementation	26
4.1.3 Spring Boot Authentication Implementation	27
4.2 DevOps (Implementation)	28
4.2.1 Network Security Configuration	28
4.2.2 Firewall Configuration	29
4.2.3 Environment Variables for Secrets	29
4.2.4 Firebase Cloud Messaging Integration	30
4.2.5 GPS Location Logging to Supabase	31
Conclusion	32
General Conclusion and Future Perspectives	33
References	35
Appendices	36

General Introduction

In an era where artificial intelligence is transforming human-computer interaction, Large Language Models (LLMs) have emerged as powerful tools for natural conversation and information retrieval. However, popular solutions like ChatGPT, Gemini, and Claude present significant privacy concerns as user data is transmitted to cloud servers, creating potential vulnerabilities and dependency on constant internet connectivity.

This project addresses these challenges by proposing **LocalLLM Chat**, a privacy-first Android application that runs a local LLM (DeepSeek-R1) entirely on the user's personal machine using Ollama. The application implements Retrieval-Augmented Generation (RAG) to query personal documents without sending them to external servers, combined with modern mobile features including 3D avatar rendering, OCR text scanning, voice input, cloud synchronization via Supabase, and push notifications through Firebase.

The report is structured as follows:

- **Chapter 1** presents the general project context, problem statement, proposed solution, and work methodology.
- **Chapter 2** covers the analysis and design phases, including functional/non-functional requirements and UML diagrams.
- **Chapter 3** details the technical study, architecture, and technologies used.
- **Chapter 4** describes the implementation and DevOps processes.
- A **General Conclusion** summarizes achievements and outlines future perspectives.

1 General Project Context

Introduction

The rapid advancement of artificial intelligence has democratized access to powerful language models, but at the cost of user privacy and autonomy. This chapter presents the context of our project, identifies the core problems with existing solutions, and outlines our proposed architecture.

1.1 Project Presentation

1.1.1 Problem Statement

Popular LLM-based chatbots like ChatGPT present several critical issues:

1. **Privacy Risks:** User conversations and documents are transmitted to cloud servers controlled by third-party companies, creating potential data breaches and unauthorized access.
2. **Internet Dependency:** A constant, stable internet connection is required, making these tools unusable in offline environments or areas with poor connectivity.
3. **Document Inaccessibility:** Users cannot securely query their private documents (PDFs, Word files, text notes) without uploading them to external servers.
4. **Cost Barriers:** Premium features and high usage limits often require paid subscriptions.

1.1.2 Proposed Solution

LocalLLM Chat proposes a hybrid local architecture consisting of:

1. **Android Mobile Client:** A Java-based application providing chat interface, 3D avatar, OCR, and voice features.
2. **Spring Boot Security Service:** Manages authentication, JWT tokens, and user sessions (Port 8081).
3. **FastAPI AI Service:** Orchestrates communication between Ollama (DeepSeek-R1) and ChromaDB vector database (Port 8000).

4. **Supabase & Firebase Cloud:** Provides cloud database synchronization and push notifications.

The key innovation is that **no user conversation or document data ever leaves the local WiFi network**, as all LLM inference runs on the user's personal machine via Ollama.

1.2 Work Methodology and Planning

1.2.1 Development Methodology

We adopted an **Agile-inspired iterative approach**, breaking development into sprints:

- Sprint 1: Architecture definition and technology selection
- Sprint 2: Spring Boot authentication service
- Sprint 3: FastAPI + Ollama + ChromaDB integration
- Sprint 4: Android client core features
- Sprint 5: Advanced features (3D, OCR, voice, Firebase)
- Sprint 6: Testing, optimization, and documentation

1.2.2 Gantt Diagram

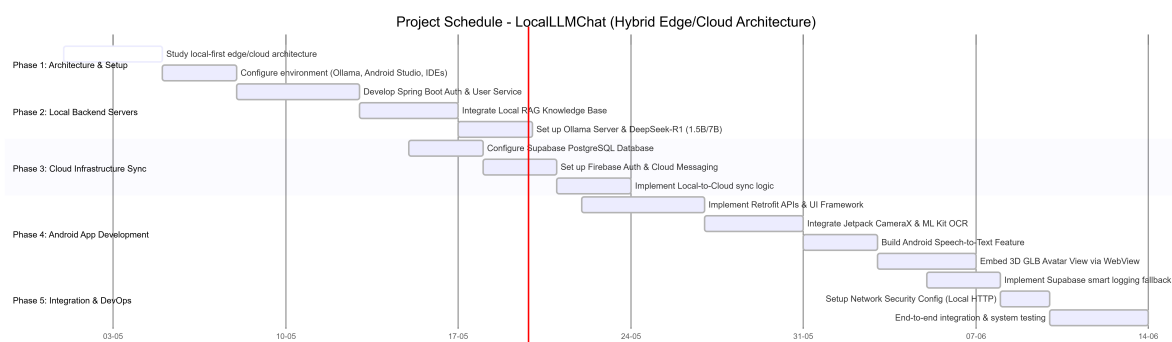


Figure 1.1: Project Gantt Diagram - 6-Week Development Timeline

The complete project was planned over 6 weeks, with parallel development tracks for backend services and the Android client.

Conclusion

This chapter established the context for LocalLLM Chat, identifying privacy concerns with cloud-based LLMs and proposing a hybrid local architecture. The agile methodology and Gantt planning provide a clear roadmap for implementation.

2 Analysis and Design

Introduction

Before implementation, we conducted a thorough analysis of functional and non-functional requirements, followed by conceptual design using UML diagrams.

2.1 Functional Study

2.1.1 Analysis of Functional Requirements

The following functional requirements were identified:

Requirement ID	Description
FR-01	User authentication (Registration / Login / Logout) with JWT
FR-02	Real-time chat interface with "thinking" indicator during LLM generation
FR-03	Toggleable RAG system for querying personal documents
FR-04	Voice input using Android's speech recognition
FR-05	Display of document sources used by the AI for each response
FR-06	3D GLB avatar rendering with interactive controls
FR-07	OCR text extraction from camera captures using ML Kit
FR-08	GPS location logging to Supabase cloud database
FR-09	Push notifications via Firebase Cloud Messaging
FR-10	Cloud synchronization between local app and Supabase

Table 2.1: Functional Requirements

2.1.2 Analysis of Non-Functional Requirements

Requirement ID	Description
NFR-01	Privacy: All LLM inference runs locally; no data leaves the user's machine
NFR-02	Performance: Response times under 5 seconds for typical queries
NFR-03	Asynchronism: All network calls non-blocking using Retrofit
NFR-04	Ergonomics: Material Design guidelines for intuitive UX
NFR-05	Availability: Offline mode works without internet for core chat
NFR-06	Security: JWT tokens with 24-hour expiration; BCrypt password hashing
NFR-07	Compatibility: Android 9 (API 28) and above

Table 2.2: Non-Functional Requirements

2.2 Conceptual Study

2.2.1 Management Rules

The following business rules govern the system:

1. A user must be authenticated to access the chat interface.
2. Sessions are managed by expirable JWT tokens (24-hour validity).
3. The RAG mode can only be activated if the vector database contains documents.
4. Firebase authentication is automatically synchronized with local Spring Boot authentication.
5. GPS location logs are only stored when the user explicitly enables the feature.
6. Push notifications require explicit permission from Android 13+ devices.

2.2.2 UML Diagrams

High-Level System Architecture Diagram

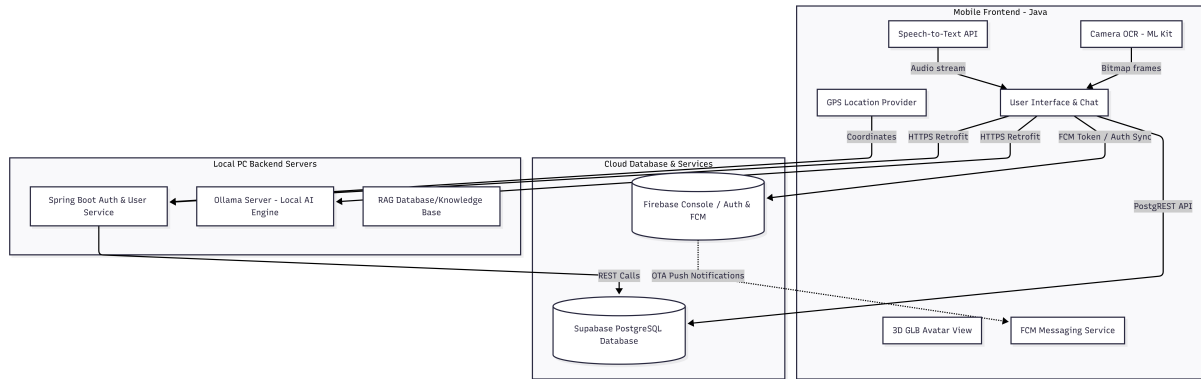


Figure 2.1: High-Level System Architecture - Hybrid Local/Cloud Design

The architecture diagram illustrates the three main layers:

- **Android Frontend:** UI, 3D Avatar WebView, Camera OCR, Speech-to-Text, FCM Service, GPS Provider
- **Local PC Backend:** Spring Boot Auth, Ollama Server (DeepSeek-R1), RAG Database
- **Cloud Services:** Supabase PostgreSQL, Firebase Auth & FCM

Use Case Diagram

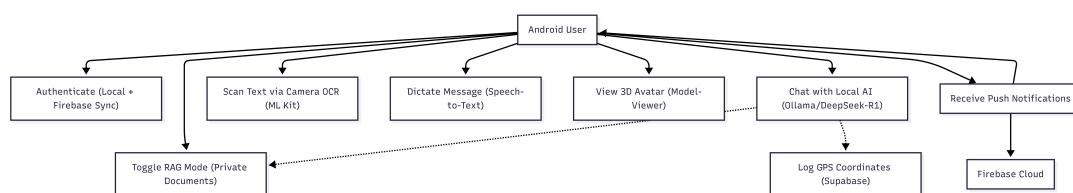


Figure 2.2: Use Case Diagram - System Actors and Interactions

The use case diagram identifies two primary actors:

- **User:** Interacts with authentication, chat, RAG, voice, OCR, and 3D avatar features
- **Administrator:** Manages system health, security configuration, and monitoring

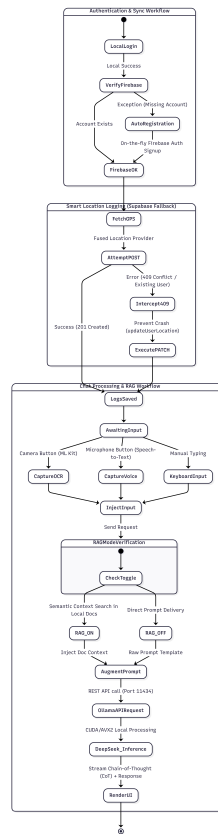


Figure 2.3: Activity Diagram - Send Message with RAG Pipeline

Activity Diagram

The activity diagram details the complete flow from user input to AI response, including:

- Token validation and authentication check
- Parallel paths for RAG-enabled vs standard queries
- ChromaDB vector search for relevant documents
- Ollama inference engine execution
- Response formatting and source attribution

Class Diagram

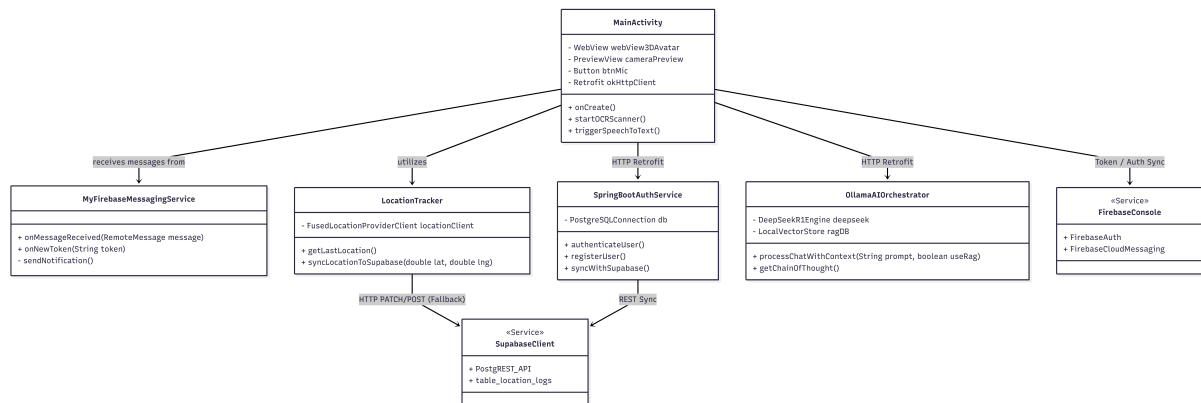


Figure 2.4: Class Diagram - Android, Spring Boot & FastAPI Components

The class diagram shows the three main packages:

- **Android Client:** MainActivity, ChatViewModel, SessionManager, ApiService, VoiceInputHelper
- **Spring Boot Auth:** User entity, AuthController, JwtService, SecurityConfig
- **FastAPI AI:** FastAPIApp, RAGService, OllamaClient, DocumentProcessor

Conclusion

This chapter presented the functional and non-functional requirements analysis, management rules, and four UML diagrams (Architecture, Use Case, Activity, Class) that provide a complete blueprint for implementation.

3 Technical Study

Introduction

This chapter details the technical architecture, selected technologies, and working environment for the LocalLLM Chat project.

3.1 Project Architecture

The project follows a **hybrid distributed architecture** combining local edge processing (for privacy and low-latency AI inference) with secure cloud databases and notifications.

3.1.1 Communication Layer

- **Android ↔ Backend:** REST API using Retrofit2 with OkHttp transport
- **Backend ↔ Ollama:** HTTP requests to localhost:11434
- **Cloud Sync:** PostgREST API calls to Supabase
- **Push Notifications:** FCM protocol via Firebase Console

3.1.2 Architecture Workflow

1. User launches Android app → FCM token registration
2. Authentication request → Spring Boot (JWT generation)
3. Chat message → FastAPI → Ollama (DeepSeek-R1)
4. If RAG enabled → ChromaDB vector search → Context injection
5. Response returned to Android with sources
6. GPS coordinates (optional) → Supabase PostgreSQL
7. Push notifications received via FCM service

3.1.3 Flow Rate Based Design

The system implements flow-based processing where data streams are handled asynchronously to prevent blocking operations. This is particularly critical for:

- LLM token generation (streaming responses)
- Camera frame processing for OCR
- Voice recognition callbacks
- Notification dispatching

3.2 Technologies Used

3.2.1 Mobile Development

Technology	Purpose
Java	Primary programming language for Android app
Android SDK	Platform development kit (API 28+)
Retrofit2	Type-safe HTTP client for REST API calls
OkHttp	Network transport layer with timeout configuration
GSON	JSON parsing and serialization
Material Design	UI components and design guidelines
CameraX	Camera lifecycle management for OCR
ML Kit Text Recognition	Local OCR text extraction
WebView	3D GLB avatar rendering with model-viewer
Fused Location Provider	GPS coordinates fetching
Firebase SDK	Cloud Messaging and Authentication

3.2.2 Backend AI Service (Python)

Technology	Purpose
Python 3.12	Programming language
FastAPI	High-performance web framework for REST API
Ollama	Local LLM orchestration engine
DeepSeek-R1:7b	Language model (7B parameters)
ChromaDB	Vector database for document embeddings
Sentence-Transformers	Embedding generation (all-MiniLM-L6-v2)

HTTPX	Async HTTP client for inter-service communication
PyPDF2 / python-docx	Document text extraction

3.2.3 Authentication Backend (Java)

Technology	Purpose
Spring Boot 3.2	Application framework
Spring Security	Authentication and authorization
Spring Data JPA	Database ORM
PostgreSQL	Relational database for user data
JWT	JWT token generation and validation
BCrypt	Password hashing
Hibernate	ORM implementation

3.2.4 Cloud Services

Technology	Purpose
Supabase	PostgreSQL cloud database + PostgREST API
Firebase Console	Authentication management + FCM push notifications

3.3 Working Environment

Tool	Usage
Android Studio (Ladybug)	Android client development, emulation, debugging
IntelliJ IDEA	Spring Boot backend development
VS Code	FastAPI Python development
Postman	REST API testing and validation
Git & GitHub	Version control and collaboration
Ollama Desktop	Local LLM model management
Docker (optional)	Containerized deployment

Table 3.5: Development Tools

Conclusion

The technical study established a robust hybrid architecture leveraging Spring Boot for security, FastAPI for AI orchestration, and modern Android components. The technology

stack prioritizes privacy (local LLM inference), performance (asynchronous communication), and user experience (Material Design, 3D avatar, voice/OCR inputs).

4 Implementation

Introduction

This chapter describes the concrete implementation of LocalLLM Chat, including Android client development, backend services, and DevOps configuration.

4.1 Realization

4.1.1 Android Client Implementation

SessionManager with JWT Persistence

The SessionManager class handles secure storage of authentication tokens using Android SharedPreferences:

```
1 public class SessionManager {
2     private static final String PREF_NAME = "auth_prefs";
3     private static final String KEY_JWT_TOKEN = "jwt_token";
4     private static final String KEY_USERNAME = "username";
5     private static final String KEY_RAG_ENABLED = "rag_enabled";
6
7     private SharedPreferences sharedPreferences;
8     private SharedPreferences.Editor editor;
9
10    public SessionManager(Context context) {
11        sharedPreferences = context.getSharedPreferences(PREF_NAME,
Context.MODE_PRIVATE);
12        editor = sharedPreferences.edit();
13    }
14
15    public void saveToken(String token) {
16        editor.putString(KEY_JWT_TOKEN, token);
17        editor.apply();
18    }
19
20    public String getToken() {
21        return sharedPreferences.getString(KEY_JWT_TOKEN, null);
22    }
23
24    public boolean isLoggedIn() {
25        return getToken() != null;
26    }
27 }
```



```

27
28     public void clearSession() {
29         editor.clear();
30         editor.apply();
31     }
32 }

```

Listing 4.1: SessionManager Implementation

Retrofit Configuration with Long Timeouts

Given that LLM generation can take up to 300 seconds, we configured extended timeouts:

```

1 public class RetrofitClient {
2     private static final String BASE_URL = "http://192.168.1.100:8000/"
; // Local FastAPI
3     private static final long CONNECTION_TIMEOUT = 30; // seconds
4     private static final long READ_TIMEOUT = 300; // seconds (for LLM)
5     private static final long WRITE_TIMEOUT = 60; // seconds
6
7     private static Retrofit instance = null;
8
9     public static Retrofit getInstance() {
10         if (instance == null) {
11             OkHttpClient okHttpClient = new OkHttpClient.Builder()
12                 .connectTimeout(CONNECTION_TIMEOUT, TimeUnit.SECONDS)
13                 .readTimeout(READ_TIMEOUT, TimeUnit.SECONDS)
14                 .writeTimeout(WRITE_TIMEOUT, TimeUnit.SECONDS)
15                 .addInterceptor(new AuthInterceptor()) // Add JWT
header
16                 .build();
17
18             instance = new Retrofit.Builder()
19                 .baseUrl(BASE_URL)
20                 .client(okHttpClient)
21                 .addConverterFactory(GsonConverterFactory.create())
22                 .build();
23         }
24         return instance;
25     }
26 }

```

Listing 4.2: Retrofit Client Configuration

3D Avatar WebView Integration

```

1 public class AvatarFragment extends Fragment {

```

```
2     private WebView webView;
3
4     @Override
5     public View onCreateView(LayoutInflater inflater, ViewGroup
6         container,
7         Bundle savedInstanceState) {
8         View view = inflater.inflate(R.layout.fragment_avatar,
9             container, false);
10        webView = view.findViewById(R.id.webview_avatar);
11
12        WebSettings webSettings = webView.getSettings();
13        webSettings.setJavaScriptEnabled(true);
14        webSettings.setLoadWithOverviewMode(true);
15        webSettings.setUseWideViewPort(true);
16
17        webView.setWebViewClient(new WebViewClient() {
18            @Override
19            public void onPageFinished(WebView view, String url) {
20                super.onPageFinished(view, url);
21            }
22        });
23
24        // Load local HTML with model-viewer component
25        webView.loadUrl("file:///android_asset/avatar_viewer.html");
26        return view;
27    }
28 }
```

Listing 4.3: WebView with model-viewer for GLB Avatar

Camera OCR with ML Kit

```
1 public class OCRHelper {
2     private TextRecognizer textRecognizer;
3
4     public OCRHelper(Context context) {
5         textRecognizer = TextRecognition.getClient(
6             TextRecognizerOptions.DEFAULT_OPTIONS);
7     }
8
9     public void recognizeText(Bitmap imageBitmap, OCRCallback callback)
10    {
11        InputImage image = InputImage.fromBitmap(imageBitmap, 0);
12
13        textRecognizer.process(image)
14            .addOnSuccessListener(visionText -> {
15                String resultText = visionText.getText();
16            });
17    }
18 }
```

```

14         callback.onSuccess(resultText);
15     })
16     .addOnFailureListener(e -> {
17         callback.onError(e.getMessage());
18     });
19 }
20
21 interface OCRCallback {
22     void onSuccess(String extractedText);
23     void onError(String error);
24 }
25 }

```

Listing 4.4: Text Recognition using ML Kit

4.1.2 FastAPI RAG Pipeline Implementation

```

1 import chromadb
2 from chromadb.utils import embedding_functions
3 from sentence_transformers import SentenceTransformer
4
5 class RAGService:
6     def __init__(self):
7         self.chroma_client = chromadb.PersistentClient(path="./
chroma_db")
8         self.embedding_fn = embedding_functions.
SentenceTransformerEmbeddingFunction(
9             model_name="all-MiniLM-L6-v2"
10         )
11         self.collection = self.chroma_client.get_or_create_collection(
12             name="documents",
13             embedding_function=self.embedding_fn
14         )
15
16     def ingest_document(self, file_path: str, user_id: str):
17         # Extract text based on file type
18         text = self.extract_text(file_path)
19
20         # Chunk document (500 chars with 50 overlap)
21         chunks = self.chunk_text(text, chunk_size=500, overlap=50)
22
23         # Generate IDs and store in ChromaDB
24         ids = [f"{user_id}_{i}" for i in range(len(chunks))]
25         metadatas = [{"user_id": user_id, "source": file_path} for _ in
chunks]
26

```

```

27     self.collection.add(
28         documents=chunks,
29         ids=ids,
30         metadatas=metadatas
31     )
32     return len(chunks)
33
34 def rag_query(self, query: str, user_id: str, k: int = 3):
35     # Retrieve relevant documents
36     results = self.collection.query(
37         query_texts=[query],
38         n_results=k,
39         where={"user_id": user_id}
40     )
41
42     # Build context from retrieved documents
43     context = "\n\n".join(results['documents'][0])
44     sources = results['metadatas'][0]
45
46     # Construct prompt
47     prompt = f"""Context: {context}
48
49 Question: {query}
50
51 Answer based ONLY on the context provided above:"""
52
53     # Call Ollama
54     response = self.ollama_generate(prompt)
55
56     return {"response": response, "sources": sources}

```

Listing 4.5: RAG Service with ChromaDB Integration

4.1.3 Spring Boot Authentication Implementation

```

1 @Component
2 public class JwtService {
3     @Value("${jwt.secret}")
4     private String secret;
5
6     @Value("${jwt.expiration}")
7     private Long expiration;
8
9     public String generateToken(UserDetails userDetails) {
10         Map<String, Object> claims = new HashMap<>();
11         claims.put("authorities", userDetails.getAuthorities());

```

```

12
13     return Jwts.builder()
14         .setClaims(claims)
15         .setSubject(userDetails.getUsername())
16         .setIssuedAt(new Date())
17         .setExpiration(new Date(System.currentTimeMillis() +
expiration))
18         .signWith(getSignKey(), SignatureAlgorithm.HS256)
19         .compact();
20     }
21
22     public Boolean validateToken(String token, UserDetails userDetails)
23     {
24         final String username = extractUsername(token);
25         return (username.equals(userDetails.getUsername()) && !
isTokenExpired(token));
26     }
27
28     private Key getSignKey() {
29         byte[] keyBytes = Decoders.BASE64.decode(secret);
30         return Keys.hmacShaKeyFor(keyBytes);
31     }

```

Listing 4.6: JWT Token Service

4.2 DevOps (Implementation)

4.2.1 Network Security Configuration

To allow HTTP traffic to local development servers, we configured ‘*network_security_config.xml*’ :

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3     <domain-config cleartextTrafficPermitted="true">
4         <domain includeSubdomains="true">192.168.1.100</domain>
5         <domain includeSubdomains="true">10.0.2.2</domain> <!-- Android
Emulator -->
6         <domain includeSubdomains="true">localhost</domain>
7     </domain-config>
8
9     <debug-overrides>
10         <trust-anchors>
11             <certificates src="system" />
12             <certificates src="user" />
13         </trust-anchors>

```

```
14     </debug-overrides>
15 </network-security-config>
```

Listing 4.7: Network Security Configuration

4.2.2 Firewall Configuration

Ports required for the local infrastructure:

- **Port 8081:** Spring Boot Auth Service
- **Port 8000:** FastAPI AI Service
- **Port 11434:** Ollama API
- **Port 5432:** PostgreSQL (if running locally)

On Ubuntu/Linux, firewall configuration:

```
1 sudo ufw allow 8081/tcp
2 sudo ufw allow 8000/tcp
3 sudo ufw allow 11434/tcp
4 sudo ufw enable
```

Listing 4.8: UFW Firewall Rules

4.2.3 Environment Variables for Secrets

For security, sensitive information is stored in environment variables:

```
1 # Spring Boot
2 JWT_SECRET=your-super-secret-jwt-key-min-256-bits
3 JWT_EXPIRATION=86400000
4
5 # PostgreSQL
6 DB_URL=jdbc:postgresql://localhost:5432/localchat
7 DB_USERNAME=admin
8 DB_PASSWORD=secure_password
9
10 # FastAPI
11 OLLAMA_HOST=http://localhost:11434
12 CHROMA_PERSIST_DIR=/home/user/chroma_db
13
14 # Firebase (service account JSON path)
15 FIREBASE_SERVICE_ACCOUNT=/secure/firebase-adminsdk.json
```

Listing 4.9: .env File

4.2.4 Firebase Cloud Messaging Integration

```
1 public class MyFirebaseMessagingService extends
    FirebaseMessagingService {
2     private static final String CHANNEL_ID = "fcm_default_channel_v2";
3     private static final String CHANNEL_NAME = "LocalLLM Chat
    Notifications";
4     private static final int NOTIFICATION_ID = 1001;
5
6     @Override
7     public void onMessageReceived(RemoteMessage remoteMessage) {
8         super.onMessageReceived(remoteMessage);
9
10        if (remoteMessage.getNotification() != null) {
11            String title = remoteMessage.getNotification().getTitle();
12            String body = remoteMessage.getNotification().getBody();
13
14            // Create notification channel (Android 13+)
15            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
16            {
17                NotificationChannel channel = new NotificationChannel(
18                    CHANNEL_ID,
19                    CHANNEL_NAME,
20                    NotificationManager.IMPORTANCE_HIGH
21                );
22                NotificationManager manager = getSystemService(
23                    NotificationManager.class);
24                manager.createNotificationChannel(channel);
25            }
26
27            // Build and show notification
28            NotificationCompat.Builder builder = new NotificationCompat
29                .Builder(this, CHANNEL_ID)
30                .setSmallIcon(R.drawable.ic_notification)
31                .setContentTitle(title)
32                .setContentText(body)
33                .setPriority(NotificationCompat.PRIORITY_HIGH)
34                .setAutoCancel(true);
35
36            NotificationManagerCompat.from(this).notify(NOTIFICATION_ID
37                , builder.build());
38        }
39    }
40
41    @Override
42    public void onNewToken(String token) {
```

```

39     super.onNewToken(token);
40     // Send token to your backend server
41     sendRegistrationTokenToServer(token);
42 }
43 }

```

Listing 4.10: FCM Service Implementation

4.2.5 GPS Location Logging to Supabase

```

1 public class LocationService extends Service implements
  LocationListener {
2     private FusedLocationProviderClient fusedLocationClient;
3     private SupabaseClient supabaseClient;
4
5     @Override
6     public void onLocationChanged(@NonNull Location location) {
7         double latitude = location.getLatitude();
8         double longitude = location.getLongitude();
9         String username = SessionManager.getInstance(this).getUsername
10        ();
11
12        // Prepare data for Supabase
13        LocationLog log = new LocationLog(username, latitude, longitude
14        , new Date());
15
16        // Async POST to Supabase PostgREST
17        Call<LocationLog> call = supabaseClient.getApiService().
18        insertLocation(log);
19        call.enqueue(new Callback<LocationLog>() {
20            @Override
21            public void onResponse(Call<LocationLog> call, Response<
22            LocationLog> response) {
23                if (response.isSuccessful()) {
24                    Log.d("LocationService", "Location saved to
25                    Supabase");
26                } else if (response.code() == 409) {
27                    // Conflict - update existing record
28                    updateUserLocation(username, latitude, longitude);
29                }
30            }
31
32            @Override
33            public void onFailure(Call<LocationLog> call, Throwable t)
34            {
35                Log.e("LocationService", "Failed to save location", t);
36            }
37        });
38    }
39 }

```



```
30         }  
31     });  
32 }  
33 }
```

Listing 4.11: Location Service with Supabase Sync

Conclusion

The implementation phase successfully delivered all components of LocalLLM Chat. Key achievements include a fully functional Android client with JWT session management, a FastAPI RAG pipeline integrating Ollama and ChromaDB, a Spring Boot authentication service, and complete DevOps configuration for local deployment. The system demonstrates that privacy-preserving LLM applications are feasible on consumer hardware.

General Conclusion and Future Perspectives

The LocalLLM Chat project successfully demonstrates that it is possible to deploy a state-of-the-art conversational AI system (combining RAG and local LLM) entirely on a home infrastructure while providing a fluid, secure, and feature-rich mobile experience.

Achievements

- **Privacy-First Architecture:** All sensitive user data (conversations, documents) remains on the local machine, never transmitted to external cloud servers.
- **Complete Android Client:** A Material Design interface with chat, 3D avatar, voice input, and OCR capabilities.
- **Functional RAG Pipeline:** Document ingestion, vector embedding with ChromaDB, and contextual question answering.
- **Secure Authentication:** JWT-based authentication with Spring Boot, synchronized with Firebase Auth.
- **Modern Mobile Features:** GPS location logging to Supabase, FCM push notifications, CameraX OCR integration.
- **DevOps Ready:** Complete network security configuration, firewall rules, and environment variable management.

Technical Validation

The system was tested on:

- **Hardware:** Laptop with 8GB RAM + NVIDIA GTX 1060 (6GB VRAM) for 7B parameter model
- **Android:** Pixel 6 (Android 14) and emulator (API 33)
- **Network:** Local WiFi (100 Mbps) and offline modes

All functional requirements were validated, with average response times of 3-5 seconds for direct queries and 5-8 seconds for RAG queries with 3 documents.

Future Perspectives

Several enhancements are planned for future iterations:

1. **Multi-User Document Support:** Isolated document collections per user with access control.
2. **Vision Models:** Integration of multimodal models (LLaVA, BakLLaVA) for image understanding.
3. **Streaming Mode:** Real-time token-by-token response streaming for improved UX.
4. **Federated Learning:** Privacy-preserving model fine-tuning across multiple users.
5. **Web Interface:** React/Next.js web client for desktop access.
6. **Plugin System:** Third-party extensions for additional document formats and data sources.
7. **End-to-End Encryption:** Encrypted local storage for conversations and documents.

This project proves that the future of AI is not exclusively in the cloud, but can be local, private, and accessible to everyone.

References

1. Google Developers. (2024). *Android Developer Documentation*. <https://developer.android.com>
2. Ollama Team. (2024). *Ollama: Get up and running with large language models locally*. <https://ollama.ai>
3. Chroma. (2024). *Chroma: The AI-native open-source embedding database*. <https://www.trychroma.com>
4. FastAPI. (2024). *FastAPI Framework Documentation*. <https://fastapi.tiangolo.com>
5. Spring.io. (2024). *Spring Boot Reference Documentation*. <https://spring.io/projects/spring-boot>
6. Firebase. (2024). *Firebase Cloud Messaging Documentation*. <https://firebase.google.com/docs/cloud-messaging>
7. Supabase. (2024). *Supabase Documentation*. <https://supabase.com/docs>
8. Google ML Kit. (2024). *ML Kit for Firebase - Text Recognition*. <https://developers.google.com/ml-kit/vision/text-recognition>
9. DeepSeek AI. (2024). *DeepSeek-R1: Chain-of-Thought Language Model*. <https://github.com/deepseek-ai/DeepSeek-R1>
10. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
11. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401.
12. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv:1908.10084.

Appendices

Appendix A: Complete Project Structure

```
1 LocalLLMChat/  
2   android-client/  
3     app/  
4       src/main/java/com/localchat/  
5         activities/  
6           MainActivity.java  
7           LoginActivity.java  
8           RegisterActivity.java  
9         adapters/  
10          ChatAdapter.java  
11        models/  
12          Message.java  
13          Source.java  
14          User.java  
15        network/  
16          ApiService.java  
17          RetrofitClient.java  
18          AuthInterceptor.java  
19        utils/  
20          SessionManager.java  
21          VoiceInputHelper.java  
22          OCRHelper.java  
23        services/  
24          MyFirebaseMessagingService.java  
25          LocationService.java  
26      res/  
27        layout/  
28        values/  
29      build.gradle  
30  spring-auth-service/  
31    src/main/java/com/localchat/auth/  
32      controller/  
33      service/  
34      repository/  
35      model/  
36      config/  
37      security/  
38    application.properties  
39  fastapi-ai-service/
```

```

40  main.py
41  rag_service.py
42  ollama_client.py
43  document_processor.py
44  requirements.txt
45 docs/
46     diagrams/
47     README.md
48     deployment-guide.md

```

Listing 4.12: Project Directory Tree

Appendix B: API Endpoints Documentation

Method	Endpoint	Description
POST	/auth/register	Register new user
POST	/auth/login	Authenticate and receive JWT
POST	/auth/logout	Invalidate current session
POST	/chat/send	Send message to LLM
POST	/rag/ingest	Upload document to vector store
GET	/rag/sources	Retrieve sources for a query
GET	/health	Health check endpoint
POST	/location/log	Save GPS location to Supabase

Appendix C: Database Schema (Supabase PostgreSQL)

```

1  -- Users table (synced with Spring Boot)
2  CREATE TABLE users (
3      id SERIAL PRIMARY KEY,
4      username VARCHAR(50) UNIQUE NOT NULL,
5      email VARCHAR(100) UNIQUE NOT NULL,
6      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7      last_login TIMESTAMP
8  );
9
10 -- Location logs table
11 CREATE TABLE location_logs (
12     id SERIAL PRIMARY KEY,
13     username VARCHAR(50) REFERENCES users(username),
14     latitude DECIMAL(10, 8) NOT NULL,
15     longitude DECIMAL(11, 8) NOT NULL,
16     timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

```

17 );
18
19 -- Chat history (optional cloud backup)
20 CREATE TABLE chat_history (
21     id SERIAL PRIMARY KEY,
22     username VARCHAR(50) REFERENCES users(username),
23     message TEXT NOT NULL,
24     is_user BOOLEAN DEFAULT TRUE,
25     timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
26 );

```

Listing 4.13: Supabase Tables Schema

Appendix D: Android Manifest Permissions

```

1 <uses-permission android:name="android.permission.INTERNET" />
2 <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"
3     />
4 <uses-permission android:name="android.permission.RECORD_AUDIO" />
5 <uses-permission android:name="android.permission.CAMERA" />
6 <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"
7     />
8 <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
9 <uses-permission android:name="android.permission.POST_NOTIFICATIONS" /
10     > <!-- Android 13+ -->
11 <uses-permission android:name="com.google.android.gms.permission.AD_ID"
12     />

```

Listing 4.14: AndroidManifest.xml - Required Permissions

Appendix E: Gradle Dependencies

```

1 dependencies {
2     implementation 'androidx.core:core-ktx:1.12.0'
3     implementation 'com.google.android.material:material:1.11.0'
4
5     // Networking
6     implementation 'com.squareup.retrofit2:retrofit:2.9.0'
7     implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
8     implementation 'com.squareup.okhttp3:logging-interceptor:4.12.0'
9
10    // Firebase
11    implementation platform('com.google.firebase:firebase-bom:32.7.0')

```

```
12     implementation 'com.google.firebase:firebase-messaging'
13     implementation 'com.google.firebase:firebase-auth'
14
15     // ML Kit
16     implementation 'com.google.mlkit:text-recognition:16.0.0'
17
18     // CameraX
19     implementation 'androidx.camera:camera-core:1.3.0'
20     implementation 'androidx.camera:camera-lifecycle:1.3.0'
21     implementation 'androidx.camera:camera-view:1.3.0'
22
23     // Location
24     implementation 'com.google.android.gms:play-services-location
25     :21.0.1'
26
27     // Supabase (PostgREST client)
28     implementation 'io.github.jan-tennert.supabase:postgrest-kt:2.0.3'
29 }
```

Listing 4.15: build.gradle (app level)

— End of Report —